DATASTOCK

Práctica integradora

Acceso a datos

2.º DAM · Curso 2026–2027

IES ISIDRA DE GUZMÁN · ALCALÁ DE HENARES

1. Qué es DataStock y cómo se desarrollará 4
Objetivo final de la aplicación 4
UT1 · Ficheros y formatos de intercambio 4
Resultado de aprendizaje y criterios evaluados en la UT1 4
UT2 · Acceso a bases de datos relacionales (JDBC) 5
Resultado de aprendizaje y criterios evaluados en la UT2 5
UT3 · Componentes de acceso a datos 5
Resultado de aprendizaje y criterios evaluados en la UT3 5
2. Punto de partida, material facilitado y trabajo del alumno 5
Material que recibirás 6
Lo que tendrás que desarrollar 6
Cómo trabajarás 6
3. Situación profesional y funcionamiento final de la aplicación 6
Un ejemplo de uso en la UT1 6
4. Interfaz de usuario y funcionamiento de DataStock en la primera fase 7
5. Modelo de datos: entidades, relaciones y restricciones 10
Relaciones del modelo de datos 10
Tabla física: Categoria 11
Tabla física: Proveedor 11
Tabla física: Producto 11
Tabla física: MovimientoStock 12
6. Formatos de intercambio y reglas comunes 13
Ejemplo JSON 13
Ejemplo XML 13
¿Qué ocurre cuando importamos un fichero que contiene errores? 13
7. Requisitos funcionales · Aplicación principal 14
RF-FIC-01. Explorar rutas, ficheros y directorios 14
¿Qué vas a conseguir con esta funcionalidad? 14
Descripción y alcance 14
Qué tienes que programar 14
Ejemplo de ejecución 15
Situaciones que debes controlar 15
Pruebas de aceptación 15
Antes de darlo por terminado, comprueba que 15
RF-FIC-02. Importar un catálogo JSON 15
¿Qué vas a conseguir con esta funcionalidad? 15

Descripción y alcance....15
Qué tienes que programar....15
Reglas particulares....16
Ejemplo de ejecución....16
Situaciones que debes controlar....16
Pruebas de aceptación....16
Antes de darlo por terminado, comprueba que....16
RF-FIC-03. Importar un catálogo XML mediante DOM....16
¿Qué vas a conseguir con esta funcionalidad?....16
Descripción y alcance....16
Qué tienes que programar....16
Ejemplo de ejecución....17
Situaciones que debes controlar....17
Pruebas de aceptación....17
Antes de darlo por terminado, comprueba que....17
RF-FIC-04. Consultar y buscar productos en memoria....17
¿Qué vas a conseguir con esta funcionalidad?....17
Descripción y alcance....17
Qué tienes que programar....17
Ejemplo de ejecución....18
Situaciones que debes controlar....18
Pruebas de aceptación....18
Antes de darlo por terminado, comprueba que....18
RF-FIC-05. Exportar el inventario a JSON y XML....18
¿Qué vas a conseguir con esta funcionalidad?....18
Descripción y alcance....18
Qué tienes que programar....18
Conversión entre formatos....18
Ejemplo de ejecución....19
Situaciones que debes controlar....19
Pruebas de aceptación....19
Antes de darlo por terminado, comprueba que....19
RF-FIC-06. Guardar y recuperar el inventario mediante serialización....19
¿Qué vas a conseguir con esta funcionalidad?....19
Descripción y alcance....19
Qué tienes que programar....19
Diferencia entre operaciones....19

Ejemplo de ejecución 20
Situaciones que debes controlar 20
Pruebas de aceptación 20
Antes de darlo por terminado, comprueba que... 20
8. Actividades técnicas complementarias obligatorias 20
RF-FIC-07. Consultas XML mediante XPath 20
¿Qué vas a conseguir con esta funcionalidad? 20
Descripción y alcance 20
Qué tienes que programar 20
Pistas XPath 21
Ejemplo de ejecución 21
Situaciones que debes controlar 21
Pruebas de aceptación 21
Antes de darlo por terminado, comprueba que... 21
RF-FIC-08. Acceso aleatorio a un fichero binario de stock 21
¿Qué vas a conseguir con esta funcionalidad? 21
Descripción y alcance 21
Qué tienes que programar 22
Cálculo de desplazamientos 22
Ejemplo de ejecución 22
Situaciones que debes controlar 22
Pruebas de aceptación 22
Antes de darlo por terminado, comprueba que... 22
9. Normas generales de desarrollo 23
10. Entrega de la UT1 y GitHub 23
11. Evaluación de la UT1: RA, CE, instrumentos y autoría 23
11.1. Qué se evalúa en esta práctica 23
11.2. Relación entre criterios de evaluación y evidencias 23
11.3. Instrumentos y pesos de evaluación 24
11.4. Autoría y defensa individual 24
11.5. Evidencias de entrega 24
12. Lista final de comprobación 24

1. Qué es DataStock y cómo se desarrollará

DataStock **es una práctica individual e integradora del módulo Acceso a Datos de 2.º DAM.**

El mismo proyecto Java se ampliará progresivamente durante tres unidades de trabajo. No son tres aplicaciones independientes: las nuevas funcionalidades aprovecharán y mejorarán lo desarrollado anteriormente.

Objetivo final de la aplicación

El propósito del proyecto es **desarrollar, de forma progresiva, una aplicación de consola** que permita a una pequeña empresa **gestionar la información de su inventario**. Al finalizar las tres unidades, el programa reunirá las siguientes funcionalidades:

- Importar catálogos de productos desde ficheros JSON y XML, comprobar los datos y comunicar las incidencias detectadas.
- Consultar los productos del inventario, buscarlos por código o nombre y exportar información en formatos reutilizables.
- Guardar y recuperar información mediante ficheros, conociendo las diferencias entre los distintos mecanismos de persistencia.
- Almacenar y gestionar categorías, proveedores, productos y movimientos de stock en una base de datos MySQL.
- Realizar altas, consultas, modificaciones y bajas mediante JDBC, y registrar operaciones de stock de forma coherente mediante transacciones.
- Organizar el acceso a los datos en componentes reutilizables, separando la presentación, las reglas de negocio y la persistencia.

UT1 · Ficheros y formatos de intercambio

En la primera unidad construiremos la base funcional de DataStock. La aplicación leerá catálogos JSON y XML, transformará sus registros en objetos Java y permitirá consultar los productos cargados en memoria.

También aprenderás a exportar el inventario, guardar y recuperar objetos serializados y controlar las incidencias que puedan producirse durante el trabajo con ficheros.

Como complemento, practicarás consultas XPath y acceso aleatorio a registros binarios en actividades independientes. En esta etapa no habrá conexión con MySQL: lo importante es comprender cómo se leen, escriben y transforman los datos, y cómo se comprueba que esas operaciones funcionan correctamente.

Resultado de aprendizaje y criterios evaluados en la UT1

RA1. *Desarrolla aplicaciones que gestionan información almacenada en ficheros, identificando el campo de aplicación de los mismos y utilizando clases específicas*.

- RA1-a. Utilización de clases para la gestión de ficheros y directorios.
- RA1-b. Acceso secuencial y aleatorio a ficheros.
- RA1-c. Recuperación de información almacenada en ficheros.
- RA1-d. Almacenamiento de información en ficheros.
- RA1-e. Conversión entre diferentes formatos de almacenamiento.
- RA1-f. Gestión de excepciones en operaciones con ficheros.
- RA1-g. Pruebas y documentación de las aplicaciones desarrolladas.

Se evaluarán los siete criterios del RA1 (a–g) mediante las actividades de ficheros, las pruebas y la documentación de esta fase.

UT2 · Acceso a bases de datos relacionales (JDBC)

Una vez que la aplicación pueda manejar catálogos mediante ficheros, en la segunda unidad incorporaremos una base de datos relacional MySQL.

Los productos dejarán de depender exclusivamente de la memoria de ejecución: se podrán recuperar y mantener mediante consultas y sentencias SQL ejecutadas desde Java con JDBC.

Se trabajará con las tablas y relaciones descritas en el modelo de datos, incluyendo la gestión de productos, proveedores, categorías y movimientos de stock.

También se abordarán las sentencias preparadas, los procedimientos almacenados y las transacciones, para que las actualizaciones se realicen con integridad. Estas funcionalidades se especificarán y entregarán en la fase de la UT2.

Resultado de aprendizaje y criterios evaluados en la UT2

RA2. *Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales, identificando y utilizando mecanismos de conexión. Se evaluará el RA2 completo, con sus once criterios de evaluación* (a–k).

- RA2-a–d: tecnologías y mecanismos de acceso, conexiones, sentencias y ejecución de consultas SQL desde Java.
- RA2-e–h: estructura de datos, recuperación de resultados, actualización de información y operaciones sobre la base de datos.
- RA2-i–k: procedimientos almacenados, transacciones y comprobación de las operaciones desarrolladas.

UT3 · Componentes de acceso a datos

Cuando ya estén implementadas las operaciones con ficheros y con MySQL, la tercera unidad se centrará en mejorar la organización interna del programa. El objetivo será convertir las soluciones desarrolladas en componentes de acceso a datos que puedan utilizarse y probarse de forma independiente.

Para ello se definirán responsabilidades claras, interfaces cuando proceda y clases de acceso a datos reutilizables, evitando mezclar las operaciones de consola con la persistencia.

El alumnado comprobará que los componentes funcionan correctamente y que pueden integrarse en la aplicación sin duplicar código.

Resultado de aprendizaje y criterios evaluados en la UT3

RA6. *Programa componentes de acceso a datos identificando las características que debe poseer un componente y utilizando herramientas de desarrollo*. En esta unidad se evaluarán únicamente los criterios a), b), c), d), h) e i), según la programación didáctica. Los restantes criterios del RA6 se trabajan en otras unidades.

- RA6-a y RA6-b: identificación de las características y herramientas de desarrollo de componentes de acceso a datos.
- RA6-c y RA6-d: diseño y programación de componentes reutilizables para las operaciones de acceso a datos.
- RA6-h y RA6-i: integración, utilización y comprobación de los componentes desarrollados.

Alcance de esta entrega: únicamente se implementará y evaluará el trabajo indicado para UT1. Las referencias a UT2 y UT3 explican hacia dónde evolucionará el proyecto, pero no añaden tareas a esta primera entrega.

La evaluación de esta fase se realizará conforme a la programación didáctica. Los resultados de aprendizaje, los criterios de evaluación, los instrumentos, sus pesos y las evidencias de autoría se explican en el apartado 11.

2. Punto de partida, material facilitado y trabajo del alumno

No empezarás con un proyecto vacío. Para que puedas centrarte en el acceso a datos, recibirás una base común de trabajo. Debes comprender esa base, pero no tendrás que programarla desde cero.

Material que recibirás

- Proyecto Maven configurado para JDK 21, con pom.xml y dependencias indicadas.
- Clases del modelo: Producto, Categoria, Proveedor y MovimientoStock.
- Menú principal de consola preparado con las opciones iniciales.
- Modelo de datos, diagrama entidad–relación y script DataStock_BD_MySQL.sql para la fase de JDBC.
- Ejemplos de catálogos y especificación de los formatos JSON y XML.

Lo que tendrás que desarrollar

- Implementar los requisitos RF-FIC-01 a RF-FIC-06 en la aplicación principal.
- Resolver RF-FIC-07 (XPath) y RF-FIC-08 (RandomAccessFile) como actividades técnicas obligatorias independientes del menú principal.
- Leer y escribir ficheros, transformar sus datos en objetos Java, validar registros y gestionar los errores previstos.
- Mantener los productos importados en una colección común durante la ejecución.
- Separar la interacción del menú de la lógica de lectura, escritura y procesamiento: no introducir la implementación completa en el switch.
- Crear y ejecutar pruebas, documentar sus resultados y mantener un historial de commits significativo en tu propio repositorio GitHub.

Cómo trabajarás

Avanza requisito a requisito. **Antes de programar, lee su descripción, identifica los datos de entrada y las reglas, y comprueba los ejemplos de ejecución**. Después implementa la funcionalidad en métodos o clases adecuados, prueba tanto los casos correctos como los errores y registra el resultado en la plantilla de pruebas. **Guarda los avances en GitHub mediante commits comprensibles**.

Las seis funcionalidades principales deben compartir la misma colección de productos. Importar JSON o XML añade los productos válidos y rechaza los duplicados; recuperar un inventario serializado sustituye la colección existente, previa confirmación. Los ejercicios complementarios de XPath y acceso aleatorio no tienen que integrarse en el menú.

3. Situación profesional y funcionamiento final de la aplicación

Una pequeña empresa dedicada a la venta de artículos informáticos trabaja con diferentes proveedores. Recibe catálogos de teclados, ratones, monitores y otros productos en formatos JSON y XML. Necesita revisar esos catálogos, detectar datos incorrectos y consultar los artículos disponibles sin depender de hojas de cálculo aisladas.

La empresa encarga una aplicación de consola llamada DataStock. En la primera fase, el personal podrá localizar ficheros de catálogos, importar productos, consultar el inventario temporal, exportar información y guardar una copia binaria para recuperarla después. Se añadirán consultas directas de XML y un ejercicio de registros de longitud fija para comprender otras técnicas de acceso a ficheros.

Más adelante, la aplicación dejará de depender únicamente de la memoria y de ficheros locales: se conectará a una base de datos MySQL, permitirá mantener los datos de manera persistente y registrar movimientos de stock. Finalmente, se reorganizará para que las operaciones de acceso a datos puedan reutilizarse y mantenerse con mayor facilidad.

Un ejemplo de uso en la UT1

1. El usuario ejecuta DataStock y selecciona «Importar catálogo JSON».
2. Introduce la ruta de un fichero recibido de un proveedor.
3. La aplicación lee el fichero, valida los registros y añade los productos correctos a la colección Java.
4. El usuario consulta los productos y localiza un teclado por su código.
5. Exporta el inventario resultante a XML para compartirlo con otro departamento.

6. Antes de salir, guarda una copia serializada; en la siguiente ejecución puede recuperarla.

En esta fase no se pide conexión a MySQL. La aplicación se utilizará desde la consola. Los ejemplos de ejecución que aparecen en los requisitos muestran el comportamiento esperado, aunque la presentación exacta pueda variar ligeramente.

4. Interfaz de usuario y funcionamiento de DataStock en la primera fase

Durante esta primera fase, DataStock será una aplicación de consola desarrollada en Java. El usuario interactuará con ella mediante un menú principal desde el que podrá acceder a las distintas funcionalidades de gestión de ficheros e inventario.

En esta unidad de trabajo todavía no se utilizará una base de datos. Los productos se almacenarán temporalmente en una colección de objetos Java compartida por las distintas funcionalidades. De esta manera, los productos importados desde un fichero JSON o XML podrán consultarse, exportarse a otro formato o guardarse mediante serialización.

Se proporcionará un proyecto Maven inicial que incluirá las clases del modelo de datos y la estructura básica del menú. El alumnado deberá completar la implementación de las funcionalidades descritas en los requisitos.

Menú principal

Al iniciar DataStock, se mostrará el siguiente menú:

```
================================================
         DATASTOCK - GESTIÓN DE INVENTARIO
                 FASE 1 - UT1
================================================
1. Explorar ficheros y directorios
2. Importar productos desde JSON
3. Importar productos desde XML
4. Consultar productos
5. Exportar productos
6. Guardar y recuperar inventario
0. Salir
------------------------------------------------
Selecciona una opción:
```

El usuario introducirá el número de la operación que desea realizar. Cuando termine, la aplicación volverá al menú principal, salvo que haya seleccionado la opción 0 para salir.

Si introduce una opción inexistente o un valor no válido, el programa deberá mostrar un mensaje informativo y permitirle volver a intentarlo, sin finalizar inesperadamente.

Opción 1. Explorar ficheros y directorios — RF-FIC-01

Esta opción permitirá introducir una ruta del sistema de archivos y consultar información sobre ella.

La aplicación comprobará si la ruta existe y si corresponde a un fichero o a un directorio. Cuando sea un directorio, mostrará su contenido inmediato, identificando los ficheros y subdirectorios que contiene.

Esta opción no tendrá submenú. Una vez finalizada la consulta, se regresará al menú principal.

Opción 2. Importar productos desde JSON — RF-FIC-02

Esta opción solicitará la ruta de un fichero JSON que contenga productos. La aplicación leerá el documento, comprobará los datos y añadirá a la colección de inventario los productos válidos que todavía no existan.

Al terminar, mostrará un resumen con el número de productos encontrados, importados y rechazados, indicando las incidencias detectadas.

Esta opción tampoco tendrá submenú.

Opción 3. Importar productos desde XML — RF-FIC-03

Su funcionamiento será similar al de la opción anterior, pero utilizando un fichero XML y las herramientas de procesamiento DOM de Java.

Los productos válidos se incorporarán a la misma colección que utilizan las demás opciones de DataStock, independientemente de su formato de origen.

Esta opción no tendrá submenú.

Opción 4. Consultar productos — RF-FIC-04

Al seleccionar esta opción se mostrará el siguiente submenú:

```
--------------------------------------------
           CONSULTA DE PRODUCTOS
--------------------------------------------
1. Listar todos los productos
2. Buscar producto por código
3. Buscar productos por nombre
0. Volver al menú principal
--------------------------------------------
Selecciona una opción:
```

4.1. Listar todos los productos. Mostrará los productos disponibles en el inventario, ordenados por código. Para cada producto se visualizarán sus datos principales, como código, nombre, precio, stock, categoría y proveedor.

4.2. Buscar producto por código. Solicitará un código y mostrará la información del producto correspondiente. Si no existe, informará al usuario.

4.3. Buscar productos por nombre. Solicitará un texto y mostrará todos los productos cuyo nombre lo contenga, sin distinguir entre mayúsculas y minúsculas.

4.0. Volver al menú principal. Permitirá abandonar el submenú sin cerrar la aplicación.

Después de ejecutar cualquiera de las tres consultas, se volverá a mostrar el submenú de consulta hasta que el usuario seleccione la opción 0.

Opción 5. Exportar productos — RF-FIC-05

Esta opción permitirá guardar los productos que se encuentran actualmente en memoria en un fichero JSON o XML.

Se mostrará el siguiente submenú:

```
------------------------------------------------
       EXPORTAR PRODUCTOS
------------------------------------------------
1. Exportar productos a JSON
2. Exportar productos a XML
0. Volver al menú principal
------------------------------------------------
Selecciona una opción:
```

5.1. Exportar productos a JSON. Solicitará la ruta de destino y generará un documento JSON con los productos del inventario.

5.2. Exportar productos a XML. Solicitará la ruta de destino y generará un documento XML con los mismos productos.

5.0. Volver al menú principal. Regresará al menú principal sin realizar ninguna exportación.

En ambas operaciones se deberán aplicar las comprobaciones indicadas en el requisito correspondiente, como verificar que existen productos para exportar y solicitar confirmación antes de sobrescribir un fichero existente.

Opción 6. Guardar y recuperar inventario — RF-FIC-06

Esta opción permitirá conservar el inventario mediante la serialización de objetos Java y recuperarlo en una ejecución posterior.

Se mostrará el siguiente submenú:

```
------------------------------------------------
      GUARDAR Y RECUPERAR INVENTARIO
------------------------------------------------
1. Guardar inventario en fichero .dat
2. Recuperar inventario desde fichero .dat
0. Volver al menú principal
------------------------------------------------
Selecciona una opción:
```

6.1. Guardar inventario. Solicitará una ruta de destino y almacenará la colección completa de productos en un fichero binario .dat mediante serialización Java.

6.2. Recuperar inventario. Solicitará la ruta de un fichero .dat previamente generado por DataStock y recuperará los productos almacenados. Esta operación sustituirá el inventario que exista en memoria, previa confirmación cuando corresponda.

6.0. Volver al menú principal. Regresará al menú principal sin guardar ni recuperar información.

Actividades complementarias obligatorias

Además de las seis opciones del menú principal, el alumnado desarrollará dos actividades independientes:

- **RF-FIC-07. Consultas XML mediante XPath.** Permitirá realizar búsquedas y consultas sobre un documento XML sin necesidad de importar previamente sus productos a DataStock.
- **RF-FIC-08. Acceso aleatorio mediante RandomAccessFile.** Permitirá trabajar con registros de longitud fija en un fichero binario, acceder directamente a una posición y modificar determinados datos.

Estas dos actividades no deberán incorporarse al menú principal de DataStock. Se desarrollarán en clases ejecutables independientes dentro del proyecto Maven.

Organización del código

El menú principal y los submenús se encargarán de presentar las opciones, recoger las respuestas del usuario y llamar a los métodos correspondientes.

No se deberá implementar toda la lógica de la aplicación dentro de las estructuras switch de los menús. Las operaciones de lectura, escritura, importación, exportación, búsqueda y serialización deberán organizarse en métodos y clases adecuados.

Esta separación facilitará la comprensión del programa y permitirá evolucionar DataStock en las siguientes unidades de trabajo, cuando se incorpore la persistencia en MySQL mediante JDBC y, posteriormente, se desarrollen componentes reutilizables de acceso a datos.

Relación entre el menú y los requisitos funcionales

Menú	Operación	Requisito
1	Explorar ficheros y directorios	RF-FIC-01
2	Importar productos desde JSON	RF-FIC-02
3	Importar productos desde XML	RF-FIC-03
4.1	Listar todos los productos	RF-FIC-04

4.2	Buscar producto por código	RF-FIC-04
4.3	Buscar productos por nombre	RF-FIC-04
5.1	Exportar productos a JSON	RF-FIC-05
5.2	Exportar productos a XML	RF-FIC-05
6.1	Guardar inventario serializado	RF-FIC-06
6.2	Recuperar inventario serializado	RF-FIC-06
Independiente	Consultas XPath	RF-FIC-07
Independiente	Acceso aleatorio con RandomAccessFile	RF-FIC-08

Nota: los números 4.1, 5.2 o 6.1 identifican las opciones dentro de sus respectivos submenús. El usuario no deberá escribir 4.1 en el menú principal: primero seleccionará 4 y después 1 en el submenú que aparezca.

5. Modelo de datos: entidades, relaciones y restricciones

La base de datos DataStock conserva el catálogo de productos, sus categorías y proveedores, así como el historial de entradas y salidas de unidades.

Cada producto pertenece a una única categoría y tiene un único proveedor.

Una categoría puede agrupar muchos productos y un proveedor puede suministrar muchos productos.

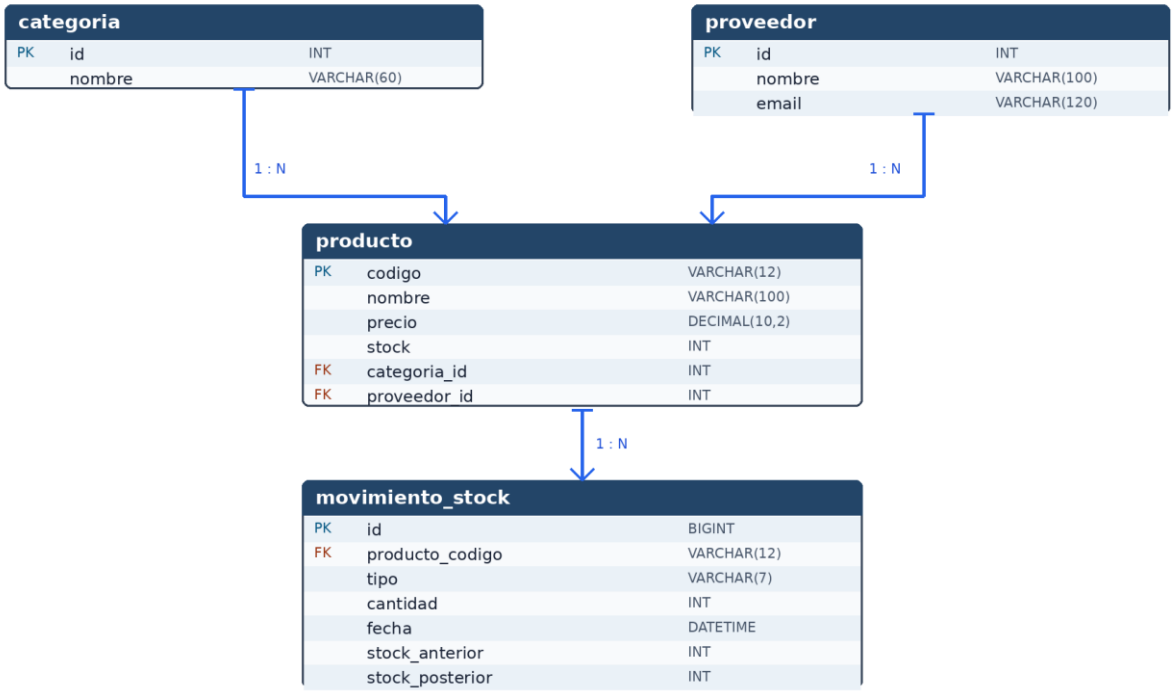
Cada movimiento de stock corresponde a un único producto; un producto puede registrar numerosos movimientos.

Las claves foráneas impiden asociar productos con categorías o proveedores inexistentes y registrar movimientos de productos no dados de alta.

Este modelo describe la estructura relacional prevista para la fase UT2. En la UT1 no debes crear estas tablas ni conectar con MySQL: trabajarás con objetos Producto en una colección en memoria. Los campos categoriaId y proveedorId se conservarán en los productos, pero su correspondencia con categorías y proveedores reales no se comprobará hasta JDBC.

Relaciones del modelo de datos

El diagrama siguiente representa el modelo entidad relación que más adelante tendrá la aplicación cuando trabaje con una base de datos MySQL.



Cada producto pertenece exactamente a una categoría y a un proveedor; una categoría o un proveedor pueden tener muchos productos. Cada movimiento corresponde a un único producto; un producto puede tener cero o muchos movimientos. Las claves foráneas impiden referencias inexistentes.

Tabla física: Categoria

Campo	Tipo SQL	Restricciones
id	INT	PK; obligatorio
nombre	VARCHAR(60)	NOT NULL; UNIQUE

Tabla física: Proveedor

Campo	Tipo SQL	Restricciones
id	INT	PK; obligatorio
nombre	VARCHAR(100)	NOT NULL
email	VARCHAR(120)	UNIQUE; puede ser NULL

Tabla física: Producto

Campo	Tipo SQL	Restricciones
codigo	VARCHAR(12)	PK; patrón [A-Z0-9-]+
nombre	VARCHAR(100)	NOT NULL; 3–100 caracteres
precio	DECIMAL(10,2)	NOT NULL; CHECK precio > 0
stock	INT	NOT NULL; CHECK stock >= 0
categoria_id	INT	NOT NULL; FK → Categoria.id
proveedor_id	INT	NOT NULL; FK → Proveedor.id

Tabla física: MovimientoStock

Campo	Tipo SQL	Restricciones
id	BIGINT	PK; AUTO_INCREMENT
producto_codigo	VARCHAR(12)	NOT NULL; FK → Producto.codigo
tipo	VARCHAR(7)	ENTRADA o SALIDA
cantidad	INT	NOT NULL; CHECK cantidad > 0
fecha	DATETIME	NOT NULL
stock_anterior	INT	NOT NULL; >= 0
stock_posterior	INT	NOT NULL; >= 0

A continuación, se aclaran algunos campos de la tabla Movimiento.

Imagina que tenemos un producto, TEC-001 (Teclado USB), con 12 unidades en el almacén.

Primero recibimos 8 unidades nuevas y, posteriormente, vendemos 5.

Movimiento	Tipo	Cantidad	stock_anterior	stock_posterior
1	ENTRADA	8	12	20
2	SALIDA	5	20	15

- stock_anterior: número de unidades disponibles inmediatamente antes de realizar la operación.
- stock_posterior: número de unidades disponibles después de realizarla.

El cálculo depende del tipo de movimiento:

- Entrada de mercancía: stock_posterior=stock_anterior+cantidad
- Salida de mercancía: stock_posterior=stock_anterior-cantidad

¿Y para qué tenemos también stock en la tabla producto?

Porque ambos cumplen funciones diferentes:

Tabla	Campo	Finalidad
producto	stock	Almacena las unidades disponibles actualmente
movimiento_stock	stock_anterior	Guarda las unidades que había antes de un movimiento
movimiento_stock	stock_posterior	Guarda las unidades que quedaron después del movimiento

Por tanto, después de las dos operaciones anteriores, el producto TEC-001 tendrá 15 unidades en producto.stock, mientras que la tabla movimiento_stock conservará los dos movimientos realizados.

¿Por qué es útil guardar ambos valores?

Porque permite:

- Consultar el historial de entradas y salidas.
- Comprobar cómo ha evolucionado el inventario.
- Detectar posibles errores en las operaciones.
- Auditar las modificaciones del stock sin perder información histórica.

6. Formatos de intercambio y reglas comunes

Durante la UT1, el inventario se mantiene en una colección de objetos Java: todavía no se crean ni consultan las tablas MySQL del apartado 5. Los identificadores de categoría y proveedor se comprueban como enteros positivos, pero su existencia en las tablas se verificará en la UT2. Los catálogos JSON tendrán un objeto raíz con la clave productos, que contiene un array. Los XML tendrán un elemento raíz catalogo y elementos producto. Los campos serán codigo, nombre, precio, stock, categoriaId y proveedorId. Ambos formatos deben producir objetos Producto equivalentes.

Ejemplo JSON

```
{
  "productos": [
    {"codigo":"TEC-001","nombre":"Teclado
USB","precio":19.95,"stock":12,"categoriaId":1,"proveedorId":1},
    {"codigo":"RAT-002","nombre":"Ratón
inalámbrico","precio":14.50,"stock":25,"categoriaId":1,"proveedorId":1}
  ]
}
```

Ejemplo XML

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
  <producto>
    <codigo>TEC-001</codigo><nombre>Teclado USB</nombre>
    <precio>19.95</precio><stock>12</stock>
    <categoriaId>1</categoriaId><proveedorId>1</proveedorId>
  </producto>
</catalogo>
```

Campo	Regla común
codigo	Obligatorio; máximo 12 caracteres; único sin distinguir mayúsculas; eliminar espacios extremos; respetar el patrón indicado en el modelo de datos
nombre	Obligatorio; entre 3 y 100 caracteres
precio	Decimal mayor que 0; usar BigDecimal en el modelo
stock	Entero mayor o igual que 0
categoriaId	Entero positivo durante UT1; integridad referencial en UT2
proveedorId	Entero positivo durante UT1; integridad referencial en UT2

¿Qué ocurre cuando importamos un fichero que contiene errores?

Cuando la aplicación importa un fichero JSON o XML, debe comprobar que su contenido puede interpretarse correctamente y que los datos de los productos cumplen las condiciones establecidas.

No todos los errores son iguales. Por eso, DataStock deberá actuar de forma diferente según el problema encontrado.

1. El fichero no tiene una estructura correcta

Puede ocurrir que el fichero contenga errores de escritura que impidan interpretar su contenido. Por ejemplo, un fichero JSON con una llave sin cerrar o un documento XML con etiquetas mal cerradas.

En este caso, la aplicación deberá informar del error y **cancelar toda la importación**. No se incorporará ningún producto del fichero y los productos que ya estuvieran cargados en la aplicación permanecerán sin cambios.

2. El fichero tiene una estructura correcta, pero algunos productos contienen datos incorrectos

Imagina que importamos un fichero con cinco productos. La aplicación consigue leerlo, pero descubre que:

- Tres productos tienen todos sus datos correctos.
- Un producto tiene un precio negativo.
- Otro producto tiene el código vacío.

En esta situación, DataStock deberá incorporar a su inventario los tres productos correctos y rechazar los otros dos, indicando por qué no se han podido importar.

Al finalizar, mostrará un resumen similar al siguiente:

```
IMPORTACIÓN DE PRODUCTOS
-------------------------------------
Productos encontrados:       5
Productos importados:        3
Productos rechazados:        2

Incidencias:
- Producto 4: el precio debe ser mayor que 0.
- Producto 5: el código es obligatorio.

Importación finalizada.
```

3. El fichero contiene un producto que ya existe en el inventario

Cada producto se identifica mediante un código único. Por tanto, si intentamos importar un producto cuyo código ya está registrado en DataStock, la aplicación deberá rechazarlo para evitar duplicados.

Por ejemplo, si ya existe el producto TEC-001 y el fichero contiene otro producto con ese mismo código, no se añadirá una segunda copia ni se sustituirán los datos del producto existente.

El programa informará de la incidencia y continuará procesando los demás productos del fichero.

En resumen: importar un fichero significa añadir productos nuevos y válidos al inventario. La importación no debe borrar ni modificar los productos que ya existen. Si el fichero no puede interpretarse, no se importa nada; si puede leerse, se incorporan únicamente los productos que cumplen las condiciones establecidas.

7. Requisitos funcionales · Aplicación principal

A continuación, se detallan todas las funcionalidades que la aplicación debe implementar

RF-FIC-01. Explorar rutas, ficheros y directorios

¿Qué vas a conseguir con esta funcionalidad?

Reconocer las rutas y ficheros con los que trabajará la aplicación.

Descripción y alcance

El usuario introduce una ruta. DataStock identifica si existe, si es fichero o directorio y muestra información básica. No es necesario recorrer subdirectorios recursivamente.

Qué tienes que programar

1. Solicitar una ruta y comprobar si existe.

2. Si es un fichero, mostrar nombre, ruta absoluta y tamaño en bytes.

3. Si es un directorio, listar sus hijos inmediatos, tipo (fichero/directorio) y tamaño cuando corresponda.

4. Mostrar el total de ficheros y subdirectorios.

5. Informar de las rutas inexistentes, vacías o inaccesibles.

Ejemplo de ejecución

```
===== EXPLORAR RUTA =====
Ruta: datos/catalogos
[FICHERO] catalogo.json      2450 bytes
[FICHERO] proveedores.xml    1380 bytes
[DIR]     antiguos
Total ficheros: 2 | Directorios: 1
```

Situaciones que debes controlar

• Ruta inexistente; fichero en vez de directorio; directorio vacío; error de acceso.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-FIC-01	Ruta con dos ficheros y un directorio	Se muestran tres elementos y totales correctos
CP-FIC-02	Ruta inexistente	Mensaje de error sin cerrar DataStock
CP-FIC-03	Ruta de un fichero	Muestra tamaño y ruta absoluta

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-a, RA1-f y RA1-g

RF-FIC-02. Importar un catálogo JSON

¿Qué vas a conseguir con esta funcionalidad?

Convertir un fichero JSON recibido de un proveedor en objetos Producto.

Descripción y alcance

Utilizarás Gson. Los productos válidos se añadirán a la colección común en memoria; los incorrectos se rechazarán con una explicación. No programes un analizador JSON manual.

Qué tienes que programar

1. Solicitar la ruta del fichero y comprobar que existe, es regular y puede leerse.

2. Leer el JSON y localizar el array productos.

3. Obtener cada registro, convertir los tipos y construir objetos Producto.

4. Validar todos los campos y comprobar que el código no existe ya en memoria ni se ha repetido en el mismo fichero.

5. Añadir los registros válidos; registrar las incidencias de los inválidos.

6. Mostrar encontrados, importados, rechazados y los motivos de rechazo.

Reglas particulares

La comparación de códigos no distingue mayúsculas; se eliminan espacios iniciales y finales. Los IDs de categoría y proveedor solo deben ser positivos en UT1. La colección no se vacía antes de importar.

Ejemplo de ejecución

```
===== IMPORTAR JSON =====
Ruta: datos/catalogo_errores.json
Productos encontrados: 5
Importados: 3 | Rechazados: 2
INCIDENCIAS:
- TEC-004: precio debe ser mayor que 0
- MON-003: código duplicado
Importación finalizada con incidencias.
```

Situaciones que debes controlar

• Fichero inexistente, directorio, sin permisos, JSON mal formado, raíz incorrecta, array ausente, campos faltantes, números inválidos, código duplicado. Si la raíz no es un objeto con el array productos, rechazar el documento completo sin modificar la colección.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-JSON-01	Tres productos válidos	Se importan los tres
CP-JSON-02	Tres válidos y dos inválidos	Se importan tres y se explican dos rechazos
CP-JSON-03	JSON mal formado	No se añade ningún producto
CP-JSON-04	Código duplicado	Se conserva el existente

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-c, RA1-f y RA1-g

RF-FIC-03. Importar un catálogo XML mediante DOM

¿Qué vas a conseguir con esta funcionalidad?

Incorporar al mismo inventario productos recibidos en formato XML.

Descripción y alcance

Usarás DOM (DocumentBuilderFactory, DocumentBuilder, Document, Element, NodeList). El resultado deberá ser equivalente a importar JSON. El analizador XML debe rechazar DTD y entidades externas para evitar XXE.

Qué tienes que programar

1. Solicitar ruta y verificar acceso.

2. Analizar el documento mediante DOM con configuración segura.

3. Comprobar que la raíz es catalogo.

4. Recorrer los elementos producto y leer los seis campos.

5. Convertir valores a tipos Java y aplicar exactamente las mismas validaciones que en JSON.

6. Añadir válidos a la colección común y explicar cada rechazo.

7. Mostrar un resumen con encontrados, importados y rechazados.

Ejemplo de ejecución

```
===== IMPORTAR XML =====
Ruta: datos/catalogo.xml
Leyendo documento XML...
Encontrados: 3 | Importados: 3 | Rechazados: 0
Importación finalizada.
```

Situaciones que debes controlar

• XML mal formado; raíz distinta; ausencia de campos; números inválidos; duplicados respecto a JSON; DTD o entidades externas; fichero inaccesible. Si la raíz no es catalogo, rechazar el documento completo sin incorporar productos.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-XML-01	XML con tres productos válidos	Se importan tres
CP-XML-02	XML con productos inválidos	Se rechazan solo los inválidos
CP-XML-03	XML con DTD	Se rechaza el documento
CP-XML-04	Código importado antes por JSON	Se rechaza el duplicado

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-c, RA1-f y RA1-g

RF-FIC-04. Consultar y buscar productos en memoria

¿Qué vas a conseguir con esta funcionalidad?

Comprobar que los productos importados están disponibles para trabajar.

Descripción y alcance

El submenú Consultar productos ofrecerá listar todos, buscar por código y buscar por fragmento del nombre. No consulta ficheros ni MySQL: usa la colección común de objetos Java.

Qué tienes que programar

1. Mostrar un submenú con las tres operaciones.

2. Listar código, nombre, precio, stock, categoría y proveedor; ordenar por código.

3. Buscar un código exacto ignorando mayúsculas y espacios extremos.

4. Buscar coincidencias parciales de nombre ignorando mayúsculas.

5. Mostrar mensajes claros si no hay productos o coincidencias.

6. No alterar la colección durante las consultas.

Ejemplo de ejecución

```
===== CONSULTAR PRODUCTOS =====
1. Listar | 2. Por código | 3. Por nombre | 0. Volver
Opción: 2
Código: tec-001
Código: TEC-001
Nombre: Teclado USB
Precio: 19.95 € | Stock: 12
Categoría: 1 | Proveedor: 1
```

Situaciones que debes controlar

• Inventario vacío; código inexistente; texto de búsqueda vacío; cero coincidencias.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-CON-01	Listar sin importar	Mensaje de inventario vacío
CP-CON-02	Buscar tec-001	Localiza TEC-001
CP-CON-03	Buscar teclado	Devuelve coincidencias parciales
CP-CON-04	Importar JSON y XML y listar	Se ven productos de ambos formatos

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-c y RA1-g (evidencia complementaria)

RF-FIC-05. Exportar el inventario a JSON y XML

¿Qué vas a conseguir con esta funcionalidad?

Compartir con otros sistemas los productos ya disponibles en DataStock.

Descripción y alcance

El submenú Exportar productos tendrá dos opciones: JSON mediante Gson y XML mediante DOM. Ambas exportan todos los productos de la colección, ordenados por código, en UTF-8 y sin alterar los datos.

Qué tienes que programar

1. Comprobar que el inventario no está vacío.

2. Solicitar ruta y comprobar que existe el directorio de destino.

3. Si existe el fichero, pedir confirmación antes de sobrescribirlo.

4. Para JSON, generar un objeto raíz con productos y escribirlo mediante Gson.

5. Para XML, construir catalogo/producto/campos mediante DOM y guardar el documento con indentación.

6. Informar de la ruta, el formato y el número de productos exportados.

Conversión entre formatos

No hay que escribir un conversor directo JSON↔XML. El modelo intermedio es la colección Java: JSON → Producto → XML, y XML → Producto → JSON.

Ejemplo de ejecución

```
===== EXPORTAR CATÁLOGO =====
1. JSON | 2. XML | 0. Volver
Opción: 1
Ruta: exportaciones/inventario.json
Fichero generado correctamente.
Productos exportados: 15
```

Situaciones que debes controlar

• Inventario vacío; directorio inexistente; falta de permisos; fichero existente y usuario cancela; error de escritura.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-EXP-01	Exportar tres a JSON	JSON válido con tres productos
CP-EXP-02	Exportar tres a XML	XML válido con tres productos
CP-EXP-03	Cancelar sobrescritura	Fichero original intacto
CP-EXP-04	Importar JSON y exportar XML	Se conservan los datos
CP-EXP-05	Exportar e importar en inventario vacío	Se recuperan los mismos productos

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-d, RA1-e, RA1-f y RA1-g

RF-FIC-06. Guardar y recuperar el inventario mediante serialización

¿Qué vas a conseguir con esta funcionalidad?

Conservar los productos entre ejecuciones sin usar todavía una base de datos.

Descripción y alcance

El submenú Guardar y recuperar inventario utilizará ObjectOutputStream y ObjectInputStream para escribir y leer una colección completa en un fichero .dat. Producto implementará Serializable en el proyecto base.

Qué tienes que programar

1. Guardar: comprobar inventario, solicitar ruta, confirmar sobrescritura y serializar la colección completa.

2. Recuperar: solicitar ruta y leer la colección desde un fichero fiable generado por DataStock.

3. Validar el tipo de objeto recuperado y sus productos.

4. Si ya hay productos en memoria, pedir confirmación antes de sustituirlos.

5. Solo sustituir la colección cuando toda la recuperación se haya completado correctamente.

6. Mostrar cantidad de productos guardados o recuperados.

Diferencia entre operaciones

Importar JSON/XML añade productos válidos; recuperar .dat sustituye el inventario. JSON y XML son formatos de intercambio legibles; la serialización Java guarda objetos binarios y no es un formato de intercambio general.

Ejemplo de ejecución

```
===== RECUPERAR INVENTARIO =====
Ruta: datos/inventario.dat
Actualmente hay 8 productos en memoria.
La recuperación sustituirá el inventario actual.
¿Continuar? (S/N): S
Inventario recuperado correctamente.
Productos disponibles: 15
```

Situaciones que debes controlar

• Inventario vacío al guardar; ruta inexistente; fichero corrupto o incompatible; cancelación; error de lectura/escritura. No deserializar ficheros de procedencia desconocida.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-SER-01	Guardar cinco, cerrar y recuperar	Se recuperan los cinco
CP-SER-02	Cancelar sustitución	Se conserva el inventario actual
CP-SER-03	Fichero dañado	Se informa y no cambia la memoria
CP-SER-04	Guardar sin productos	No se crea fichero

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-b, RA1-c, RA1-d, RA1-f y RA1-g

8. Actividades técnicas complementarias obligatorias

Las siguientes actividades forman parte de la misma entrega, pero no necesitan aparecer en el menú principal. Se realizarán como programas o clases independientes dentro del proyecto Maven, con ficheros de prueba y evidencias de ejecución.

RF-FIC-07. Consultas XML mediante XPath

¿Qué vas a conseguir con esta funcionalidad?

Consultar directamente un catálogo XML sin importar sus productos a la colección Java.

Descripción y alcance

Usarás DOM y XPath (XPathFactory, XPath, XPathConstants) para buscar por código, filtrar por categoría, filtrar por precio superior a un valor y contar productos. El fichero XML se analiza con protección frente a DTD y entidades externas.

Qué tienes que programar

1. Solicitar ruta y abrir un XML con raíz catalogo.

2. Presentar cuatro consultas: código, categoría, precio superior y contador.

3. Ejecutar las búsquedas con expresiones XPath y mostrar los nodos obtenidos.

4. Validar los valores introducidos y mostrar mensajes si no hay coincidencias.

5. Contar los elementos producto aunque alguno contenga datos incorrectos.

6. Permitir varias consultas sobre el documento ya abierto.

Pistas XPath

Expresiones orientativas: /catalogo/producto; /catalogo/producto[categoriaId=1]; /catalogo/producto[precio > 100]; count(/catalogo/producto). No concatenes sin control textos del usuario en una expresión XPath; para el código puedes seleccionar nodos y comparar su contenido en Java.

Ejemplo de ejecución

```
===== CONSULTAS XPATH =====
Fichero: datos/catalogo.xml
1. Por código | 2. Por categoría | 3. Precio superior | 4. Contar
Opción: 3
Precio mínimo: 100
MON-003   Monitor 24 pulgadas   149.99 €
Productos encontrados: 1
```

Situaciones que debes controlar

• XML mal formado; fichero inexistente; DTD; categoría o precio inválido; cero coincidencias.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-XPATH-01	Buscar código existente	Se encuentra el producto
CP-XPATH-02	Categoría 1	Solo productos de categoría 1
CP-XPATH-03	Precio superior a 100	Excluye exactamente 100
CP-XPATH-04	Contar tres productos	Resultado 3
CP-XPATH-05	XML con DTD	Se rechaza

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-c, RA1-f y RA1-g

RF-FIC-08. Acceso aleatorio a un fichero binario de stock

¿Qué vas a conseguir con esta funcionalidad?

Aprender a localizar y modificar un registro sin leer todos los anteriores.

Descripción y alcance

Crearás un fichero binario didáctico de registros de longitud fija a partir de una copia de los productos disponibles en la colección Java en el momento de generarlo. Una vez creado, stock.bin es independiente del inventario principal: las modificaciones realizadas mediante RandomAccessFile NO cambian los objetos Producto en memoria, y los cambios posteriores en memoria tampoco actualizan automáticamente stock.bin. Cada registro tendrá 12 caracteres Java para el código (24 bytes), un double para el precio (8 bytes) y un int para stock (4 bytes): 36 bytes por registro. En el modelo Producto el precio sigue siendo BigDecimal.

Qué tienes que programar

1. Generar stock.bin a partir de una copia de los productos actualmente cargados en memoria, ordenados por código; si el inventario está vacío, informar y no generar el fichero. Confirmar sobrescritura y truncar el fichero antes de escribir.

2. Escribir cada código con exactamente 12 caracteres (relleno con espacios), precio double y stock int.

3. Consultar un registro por posición lógica empezando en cero usando seek(posición × 36).

4. Modificar solo el stock con seek(posición × 36 + 32), sin reescribir el registro completo.

5. Mostrar todos los registros mediante lectura secuencial.

6. Comprobar que el tamaño del fichero es múltiplo de 36 y controlar posiciones inválidas.

Ejemplo: si al generar stock.bin un producto tiene 8 unidades y después cambias ese registro a 15 mediante RandomAccessFile, el fichero guardará 15, pero el objeto Producto en memoria seguirá teniendo 8. No se exige sincronizar ambos almacenes.

Cálculo de desplazamientos

Inicio del registro = posición × 36. Campo stock = posición × 36 + 32. Es obligatorio usar seek() para la consulta directa y la modificación; recorrer el fichero desde el principio no demuestra acceso aleatorio.

Ejemplo de ejecución

```
===== ACCESO ALEATORIO =====
Fichero: datos/stock.bin
Registros: 4 | Tamaño: 144 bytes
Consultar posición (0-3): 2
MON-003 | 149.99 € | stock 8
Nuevo stock: 15
Actualización correcta: stock 8 → 15
```

Situaciones que debes controlar

• Fichero inexistente o tamaño inválido; posición negativa o fuera de rango; stock negativo; error de lectura o escritura. El cambio en stock.bin NO modifica el inventario en memoria. La generación requiere productos en memoria; el fichero binario es una copia independiente, no una vista sincronizada del inventario.

Pruebas de aceptación

Prueba	Acción / entrada	Resultado esperado
CP-RAF-01	Cuatro registros	Fichero de 144 bytes
CP-RAF-02	Consultar posición 2	Acceso directo al tercer registro
CP-RAF-03	Modificar stock 8 a 15	Solo cambia ese campo
CP-RAF-04	Posición fuera de rango	Se rechaza
CP-RAF-05	Tamaño no múltiplo de 36	Se informa de fichero inválido

Antes de darlo por terminado, comprueba que…

☐ La operación funciona con los datos de ejemplo.

☐ Los errores previstos no cierran inesperadamente la aplicación.

☐ La actividad dispone de una clase ejecutable independiente y no se integra en el menú principal.

☐ Has probado la operación y documentado los resultados.

Criterios de evaluación relacionados: RA1-a, RA1-b, RA1-c, RA1-d, RA1-f y RA1-g

9. Normas generales de desarrollo

• Proyecto individual, Java 21 y Maven. Mantener la estructura inicial proporcionada.

• Gson para JSON, DOM para XML, XPath para consultas y RandomAccessFile para registros fijos.

• El switch de los menús solo gestiona la interacción y delega en métodos o clases; no contiene el procesamiento completo.

• Controlar excepciones sin ocultarlas silenciosamente ni terminar inesperadamente el programa.

• Cerrar flujos con try-with-resources cuando corresponda.

• Utilizar nombres comprensibles, comentarios explicativos y separar responsabilidades.

• Los ejemplos de salida son orientativos en presentación, pero los datos, las reglas y el comportamiento exigidos son obligatorios.

10. Entrega de la UT1 y GitHub

• Repositorio GitHub propio con código fuente Maven y pom.xml.

• README con instrucciones de ejecución, requisitos, opciones de menú y estructura del proyecto.

• Carpetas con ejemplos JSON, XML, ficheros de prueba válidos e inválidos y resultados de las actividades complementarias.

• Commits significativos durante el desarrollo; no se admite un único commit final como evidencia suficiente de proceso.

• Etiqueta Git para identificar la entrega de UT1

• Documento breve de pruebas: identificador CP, datos usados, resultado esperado, resultado obtenido y estado.

• No subir credenciales, ficheros temporales del IDE ni datos privados; configurar .gitignore.

11. Evaluación de la UT1: RA, CE, instrumentos y autoría

11.1. Qué se evalúa en esta práctica

Esta primera fase de DataStock permite obtener evidencias de los siete criterios de evaluación del RA1: «Desarrolla aplicaciones que gestionan información almacenada en ficheros, identificando el campo de aplicación de los mismos y utilizando clases específicas».

No se valorará únicamente que el programa funcione con los ejemplos. También será importante comprender el código, explicar las decisiones tomadas y comprobar su comportamiento con datos y situaciones diferentes.

11.2. Relación entre criterios de evaluación y evidencias

• RA1-a: exploración de ficheros y directorios y gestión de rutas (RF-FIC-01).

• RA1-b: acceso secuencial y aleatorio a ficheros; lectura de registros con RandomAccessFile y serialización (RF-FIC-06 y RF-FIC-08).

• RA1-c: recuperación de información de JSON, XML y ficheros serializados; consultas XPath y lectura de registros (RF-FIC-02, 03, 06, 07 y 08).

• RA1-d: almacenamiento de información mediante exportación, serialización y escritura binaria (RF-FIC-05, 06 y 08).

• RA1-e: transformación de datos entre JSON, objetos Java y XML (RF-FIC-02, 03 y 05).

• RA1-f: gestión de excepciones, validaciones y tratamiento de errores durante las operaciones con ficheros (requisito transversal).

• RA1-g: diseño, ejecución y documentación de pruebas, README y comprobación del funcionamiento (requisito transversal).

Un mismo requisito puede aportar evidencias de varios criterios, y un mismo criterio puede comprobarse mediante distintas funcionalidades.

11.3. Instrumentos y pesos de evaluación

Según la programación didáctica, el RA1 tiene un peso del 20 % en la calificación del módulo de Acceso a Datos. La evaluación del RA1 se realizará mediante los siguientes instrumentos:

• Práctica de desarrollo (60 %): cumplimiento de requisitos, corrección de las operaciones, tratamiento de errores, organización del código, pruebas y documentación.

• Prueba individual teórico-práctica (40 %): comprobación individual de los conocimientos y destrezas adquiridos en la unidad.

La valoración de cada criterio seguirá lo establecido en la programación didáctica. Las pruebas de aceptación del enunciado son evidencias para preparar y verificar la práctica; no sustituyen la prueba individual.

11.4. Autoría y defensa individual

La práctica es individual. Cada alumno deberá desarrollar su propia solución y ser capaz de explicar el código entregado. Durante la revisión o defensa, la profesora podrá solicitar:

• Ejecutar una funcionalidad con un fichero diferente del utilizado durante el desarrollo.

• Explicar cómo se importan, validan o exportan los productos y localizar los métodos responsables.

• Justificar el tratamiento de excepciones, interpretar una incidencia o explicar una prueba de aceptación.

• Realizar una pequeña modificación, corregir un error o comentar la evolución del proyecto a partir de sus commits.

Una aplicación que aparentemente funciona no garantiza por sí sola la valoración positiva de las evidencias: es necesario demostrar que se comprende el trabajo realizado.

11.5. Evidencias de entrega

El repositorio deberá contener el código fuente, los ficheros de ejemplo y de prueba, las actividades complementarias, el README y el registro de pruebas. Para cada caso de prueba se indicarán identificador, datos utilizados, resultado esperado, resultado obtenido y estado.

La fecha de entrega, la etiqueta Git y las instrucciones concretas de presentación se comunicarán a través del aula virtual.

12. Lista final de comprobación

☐ Los seis requisitos principales funcionan desde la aplicación DataStock.

☐ XPath y RandomAccessFile están implementados y probados como actividades complementarias.

☐ Los ficheros JSON y XML cumplen el formato indicado.

☐ Se gestionan los duplicados y los errores sin perder datos correctos.

☐ Se puede guardar, cerrar y recuperar el inventario serializado.

☐ La aplicación no mezcla toda la lógica de acceso a datos con el switch del menú.

☐ Las pruebas de aceptación tienen resultados documentados.

☐ El repositorio GitHub contiene las entregas, README y commits de desarrollo.